CO-4 ASSIGNMENT

Name: G R Aakaash **Course Code:** CSA0201

Reg No: 192412068 **Course Name:** C-Programming

TASK-1

CODE:

```
#include <stdio.h>
#include <string.h>

struct Date {
    int day, month, year;
};

struct BloodPressure {
    int systolic, diastolic;
};

struct Patient {
    char name[50];
    int age;
    char phone[11];
    struct BloodPressure bp;
    struct Date visitDate;
};

void addPatient(struct Patient *p, const char n[], int age,
                const char phone[], int sys, int dia,
                struct Date d) {
    strcpy(p->name, n);
    p->age = age;
    strcpy(p->phone, phone);
    p->bp.systolic = sys;
    p->bp.diastolic = dia;
    p->visitDate = d;
}

int main(void) {
    struct Patient p;
    struct Date d = {12, 11, 2026};

    addPatient(&p, "Arun", 25, "9876543210", 120, 80, d);

    printf("Name: %s\nAge: %d\nPhone: %s\n", p.name, p.age, p.phone);
    printf("BP: %d/%d\n", p.bp.systolic, p.bp.diastolic);
    printf("Visit Date: %02d/%02d/%04d\n",
          p.visitDate.day, p.visitDate.month, p.visitDate.year);
```

```
    return 0;
}
```

OUTPUT

```
Name: Arun
Age: 25
Phone: 9876543210
BP: 120/80
Visit Date: 12/11/2026
```

TASK-2

CODE

```
#include <stdio.h>

struct date { int day, month, year; };
struct address { char city[20]; int pincode; };
struct parcel { int id; float weight; struct address to; struct date booked; };

void print_parcel(const struct parcel *p) {
    printf("%d %.1f kg %s-%d %02d/%02d/%04d\n",
           p->id, p->weight, p->to.city, p->to.pincode,
           p->booked.day, p->booked.month, p->booked.year);
}

void sort_by_weight(struct parcel p[], int n) {
    int i, j, min;
    struct parcel temp;
    for (i = 0; i < n - 1; i++) {
        min = i;
        for (j = i + 1; j < n; j++)
            if (p[j].weight < p[min].weight)
                min = j;
        if (min != i) {
            temp = p[i];
            p[i] = p[min];
            p[min] = temp;
        }
    }
}

int main(void) {
    struct parcel list[3] = {
        {501, 4.5f, {"Madurai", 625001}, {12, 10, 2026}},
        {502, 1.2f, {"Chennai", 600077}, {12, 10, 2026}},
        {503, 2.8f, {"Coimbatore", 641001}, {13, 10, 2026}}
    };
    sort_by_weight(list, 3);
    for (int i = 0; i < 3; i++)
        print_parcel(&list[i]);
    return 0;
}
```

OUTPUT

```
502 1.2 kg Chennai-600077 12/10/2026
503 2.8 kg Coimbatore-641001 13/10/2026
501 4.5 kg Madurai-625001 12/10/2026
```

TASK-3

CODE:

```
#include <stdio.h>

enum kind { TEMPERATURE, HUMIDITY, STATUS };

struct reading {
   enum kind type;
   union {
      float celsius;
      int percent;
      char status[8];
   } value;
};

void show(const struct reading *r) {
   switch (r->type) {
      case TEMPERATURE:
         printf("Temp %.1f C\n", r->value.celsius);
         break;
      case HUMIDITY:
         printf("Humidity %d %%\n", r->value.percent);
         break;
      case STATUS:
         printf("Status %s\n", r->value.status);
         break;
   }
}

int main(void) {
   struct reading log[3] = {
      {TEMPERATURE, {.celsius = 31.5f}},
      {HUMIDITY, {.percent = 68}},
      {STATUS, {.status = "OK"}}
   };

   printf("sizeof union = %zu, sizeof struct reading = %zu\n",
         sizeof log[0].value, sizeof log[0]);
   for (int i = 0; i < 3; i++)
      show(&log[i]);
   return 0;
}
```

OUTPUT

```
sizeof union = 8, sizeof struct reading = 12
Temp 31.5 C
Humidity 68 %
Status OK
```

TASK-4

CODE:

```
#include <stdio.h>

struct account {
    int number;
    float balance;
};

void addInterest(struct account *a, float rate) {
    printf("Address inside function: %p\n", (void *)a);
    a->balance = a->balance + a->balance * rate / 100;
}

int main(void) {
    struct account list[3] = {
        {101, 5000}, {102, 12000}, {103, 800}
    };
    struct account *p = list;

    printf("Address in main: %p\n", (void *)p);
    for (int i = 0; i < 3; i++)
        addInterest(&list[i], 4);

    printf("\nUpdated Accounts:\n");
    for (int i = 0; i < 3; i++)
        printf("%d %.2f\n", p[i].number, p[i].balance);

    return 0;
}
```

OUTPUT

```
Address in main: (address of first account)
Address inside function: (address of first account)
Address inside function: (address of second account)
Address inside function: (address of third account)

Updated Accounts:
101 5200.00
102 12480.00
103 832.00
```

.

TASK-5

CODE:

```
#include <stdio.h>

int next_token(void) {
    static int token = 1000;
    return ++token;
}

int auto_token(void) {
    int token = 1000;
    return ++token;
}

int main(void) {
    int s1 = next_token(), s2 = next_token(), s3 = next_token();
    int a1 = auto_token(), a2 = auto_token(), a3 = auto_token();

    printf("static: %d %d %d\n", s1, s2, s3);
    printf("auto : %d %d %d\n", a1, a2, a3);
    return 0;
}
```

SAMPLE OUTPUT

```
static: 1001 1002 1003
auto : 1001 1001 1001
```

Stage 3 and 4 – Structures and Functions

```
#include <stdio.h>
#include <string.h>

#define MAX_ITEMS 50
#define MAX_SALES 100

struct Date {
    int day, month, year;
};

struct Packaged {
    struct Date expiry;
};

struct Cooked {
    int hour, minute;
};

struct Combo {
    int itemId[3];
    int quantity[3];
    int count;
};
```

```
enum ItemType { PACKAGED, COOKED, COMBO };

struct Item {
    int id;
    char name[30];
    float price;
    int stock;
    enum ItemType type;
    union {
        struct Packaged packaged;
        struct Cooked cooked;
        struct Combo combo;
    } detail;
};

struct Sale {
    int itemId;
    char name[30];
    int quantity;
    float revenue;
};

void show_expiry(const struct Item *item, struct Date today);
int record_sale(struct Item items[], int n, int id, int qty,
                struct Sale sales[], int *saleCount);
void sort_sales(struct Sale sales[], int n);
```

Stage 5 – Complete Working Program

```
#include <stdio.h>
#include <string.h>

#define MAX_ITEMS 10
#define MAX_SALES 50

struct Date { int day, month, year; };
struct Packaged { struct Date expiry; };
struct Cooked { int hour, minute; };
struct Combo { int itemId[3], quantity[3], count; };

enum ItemType { PACKAGED, COOKED, COMBO };

struct Item {
    int id;
    char name[30];
    float price;
    int stock;
    enum ItemType type;
    union {
        struct Packaged packaged;
        struct Cooked cooked;
        struct Combo combo;
    } detail;
};
```

```
struct Sale {
    int itemId;
    char name[30];
    int quantity;
    float revenue;
};

int record_sale(struct Item items[], int n, int id, int qty,
                struct Sale sales[], int *saleCount) {
    int i;
    for (i = 0; i < n; i++) {
        if (items[i].id == id) {
            if (qty <= 0 || items[i].stock < qty) {
                printf("Sale rejected: insufficient stock\n");
                return 0;
            }
            items[i].stock -= qty;
            sales[*saleCount].itemId = id;
            strcpy(sales[*saleCount].name, items[i].name);
            sales[*saleCount].quantity = qty;
            sales[*saleCount].revenue = items[i].price * qty;
            (*saleCount)++;
            printf("Sale successful: %s x %d\n", items[i].name, qty);
            return 1;
        }
    }
    printf("Item not found\n");
    return 0;
}

void expiry_warning(struct Item items[], int n, struct Date today) {
    int i;
    for (i = 0; i < n; i++) {
        if (items[i].type == PACKAGED) {
            struct Date d = items[i].detail.packaged.expiry;
            int days = (d.year - today.year) * 365 +
                       (d.month - today.month) * 30 +
                       (d.day - today.day);
            if (days >= 0 && days <= 2)
                printf("Expiry warning: %s expires within 2 days\n",
                       items[i].name);
        }
    }
}

void sort_sales(struct Sale sales[], int n) {
    int i, j;
    struct Sale temp;
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (sales[j].revenue < sales[j + 1].revenue) {
                temp = sales[j];
                sales[j] = sales[j + 1];
                sales[j + 1] = temp;
            }
        }
    }
```

```
}

int main(void) {
    struct Item items[3] = {
        {1, "Juice", 30, 10, PACKAGED, {.packaged = {{14, 11, 2026}}}},
        {2, "Sandwich", 50, 5, COOKED, {.cooked = {10, 30}}},
        {3, "ComboMeal", 100, 3, COMBO,
         {.combo = {{1, 2, 0}, {1, 1, 0}, 2}}}
    };
    struct Sale sales[MAX_SALES];
    int saleCount = 0;
    struct Date today = {12, 11, 2026};

    record_sale(items, 3, 1, 2, sales, &saleCount);
    record_sale(items, 3, 2, 8, sales, &saleCount);
    record_sale(items, 3, 3, 1, sales, &saleCount);

    expiry_warning(items, 3, today);
    sort_sales(sales, saleCount);

    printf("\nSales Summary (highest revenue first):\n");
    for (int i = 0; i < saleCount; i++)
        printf("%s  Qty=%d  Revenue=%.2f\n",
               sales[i].name, sales[i].quantity, sales[i].revenue);

    printf("\nRemaining Stock:\n");
    for (int i = 0; i < 3; i++)
        printf("%s: %d\n", items[i].name, items[i].stock);

    return 0;
}
```

OUTPUT:

```
Sale successful: Juice x 2
Sale rejected: insufficient stock
Sale successful: ComboMeal x 1
Expiry warning: Juice expires within 2 days

Sales Summary (highest revenue first):
ComboMeal  Qty=1  Revenue=100.00
Juice  Qty=2  Revenue=60.00

Remaining Stock:
Juice: 8
Sandwich: 5
ComboMeal: 2
```

.